Dependency-aware execution of the discovery loop

Each discovery step selects parents from an archive of evaluated candidates. A frozen strategist proposes several strategies for each parent, and a trainable coder generates strategy-conditioned programs in cross-prompt batches. Parent chains are processed concurrently; within a chain, later strategies can condition on earlier proposals. Each strategy first receives a pilot batch. Once all pilot evaluations for a parent are available, a strategy-level bandit assigns that parent's remaining rollout budget using the observed outcomes and estimated opportunity for improvement. These allocations form batches of phase-2 programs; the allocation calculation does not require generating and evaluating one additional program at a time. Pilot and phase-2 samples both contribute to coder training.

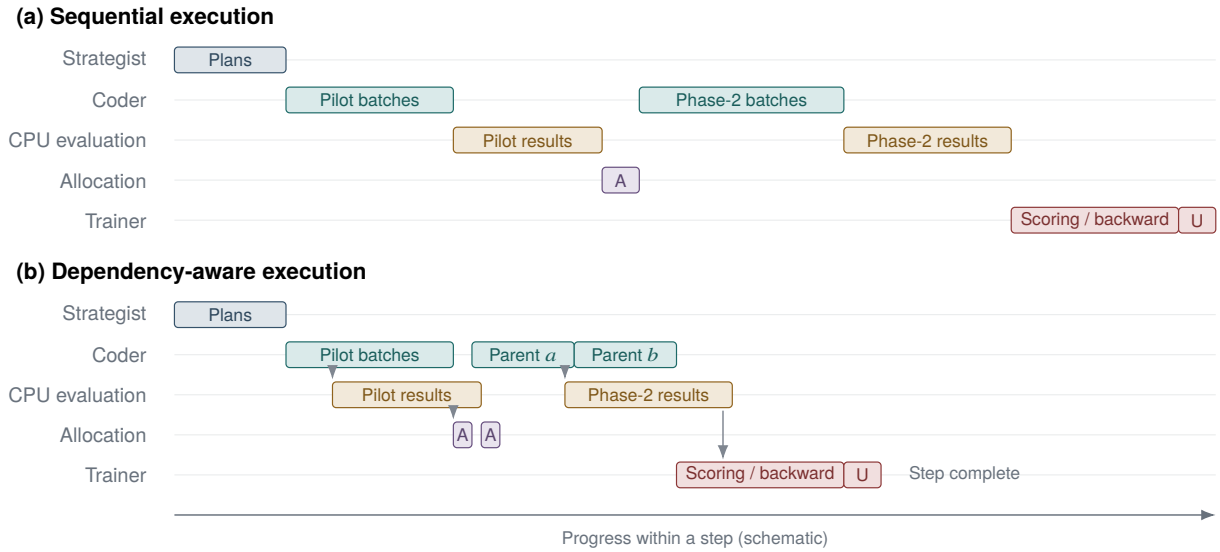


Figure 1: Overlap execution, preserve dependencies. Sequential scheduling waits for each entire stage. The implemented schedule evaluates completed programs during generation, allocates phase 2 when a parent's pilot results are ready, and overlaps training with the remaining evaluations. Parent a and parent b denote phase-2 batches; A denotes allocation and U the final optimizer update. Bar lengths are illustrative, not measured runtimes or a speedup claim.

Evaluation is driven by completion rather than by a global generation barrier. As each completed program is returned by the generator, it is saved and submitted to a CPU execution and verification pool. Evaluation therefore overlaps subsequent rollout generation in both phases. A ready parent's phase-2 work can be dispatched while other parents' pilots are still being evaluated. The program's parent, strategy, prompt, generated tokens, and evaluation outcome remain associated throughout the pipeline.

Once rollout generation and required policy scoring finish, the shared GPUs transition to training without waiting for the entire CPU evaluation queue to drain. Training inputs are prepared, and each reward-ready sample enters the distributed work queue for SPO-RS loss evaluation and backward computation. The historical normalization used to form its advantage is held fixed for the step, so unrelated pending rewards need not delay this work. All rollouts and training forwards use the same pre-update coder policy. Gradients accumulate without changing model parameters; the optimizer update is applied only after all required rewards and gradients are available and the rollout artifacts have been saved. The strategist remains frozen.

The evaluated candidates update the search archive and parent–child tree. The completed step saves the updated coder and search state for the next iteration. When an LLM-based parent selector is enabled, its next-step request can run during the remaining training work, using the newly evaluated tree. Thus, concurrency removes avoidable waiting between independent tasks without removing the data dependencies that define allocation, training, or parent selection.